INT161 · Basic Backend Development

Database Connection

Connect Node.js to MySQL with a pool, stop SQL injection with placeholders, and write the CRUD repository.

Database Setup | mysql2 and the Pool | Prepared Statements | The Repository | Reading the Body and Writing the Response | CRUD over HTTP | Glossary

01

Database Setup

Last class the data lived in an array in memory. Today it moves into a real database. The goal: do **CRUD** against MySQL from **Node**.

- A **back end** works as the middleman. The **front end** sends a method, the back end turns it into SQL for the **DBMS**, then sends the result back as **JSON**.
- The key step is a **connection**. No connection, no SQL. Once you can connect, CRUD is the easy part.
- Optional: the WebStorm database tool lets you connect and run SQL inside the editor, so you do not switch to Workbench. Host **localhost**, **port** 3306, your user and password. Press Test Connection. Success means you are set.
- Connect without a database name first, because you are about to create one.

Create the database and table

```
create database sampledb;
use sampledb;
create table subjects (
  id int primary key auto_increment,
  subject_code varchar(8) not null unique,
  subject_title varchar(60),
  credit int not null
);
insert into subjects (subject_code, subject_title, credit)
values ('INT 100', 'IT Fundamentals', 3),
       ('INT 101', 'Programming Fundamentals', 3),
       ('INT 102', 'Web Technology', 1);
```

- id is a **surrogate key**: an integer, **primary key**, **AUTO_INCREMENT**. The database counts it for you, so you never send an id on INSERT and it never repeats.
- subject_code is NOT NULL and UNIQUE. A second INT 200 will be refused.
- CHAR or VARCHAR? If every value has exactly 8 characters, CHAR(8) is the better fit. VARCHAR carries overhead for values of varying length.
- Copying SQL from a PowerPoint slide can bring in hidden line-break symbols. Find and replace them with nothing, then reformat the code.

> ⚠ **Careful with WHERE**
>
> UPDATE and DELETE without a WHERE change every row in the table. The lecturer has seen data lost this way. Always check the WHERE before you run it.

02

mysql2 and the Pool

Node needs its own **driver**. The driver in WebStorm is only for the editor and does not count. Install mysql2 in the project. Use the terminal inside the IDE so you are already in the project folder.

project root

```
$ npm install mysql2
added 12 packages, and audited 13 packages
# a node_modules folder appears, and package.json lists the dependency
```

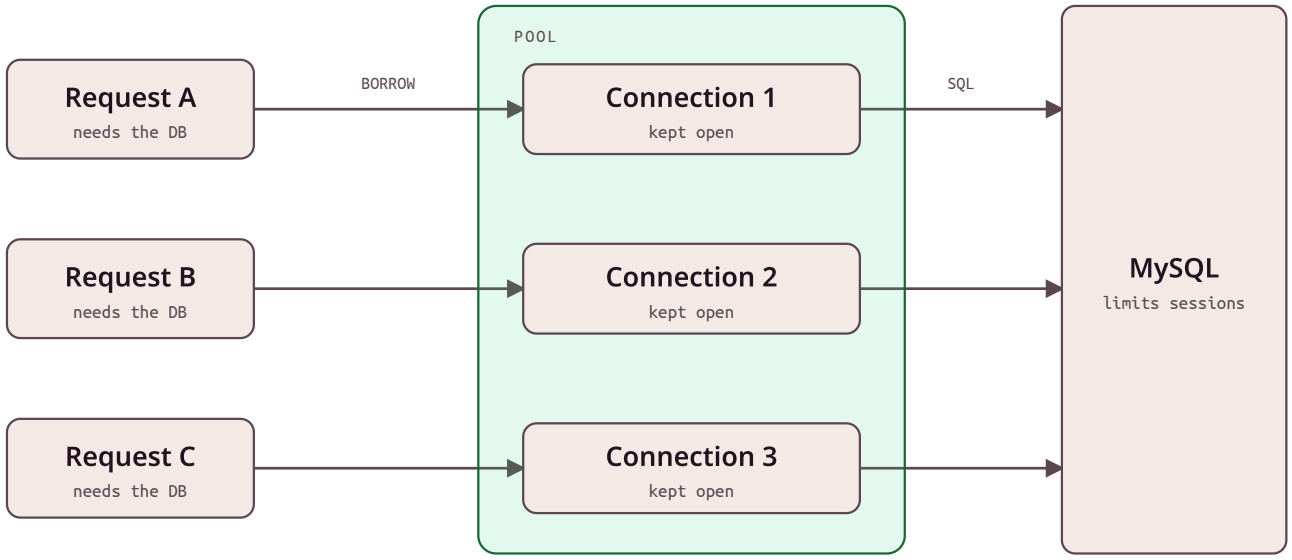
- Use the driver that matches your database and its version. A wrong one may fail to connect. We use mysql2 for MySQL. PostgreSQL and MongoDB have their own drivers.
- The driver is **async** and non-blocking, so it fits the **event loop**.

Connection

A new connection per **request**. Hundreds of **clients** may overwhelm the database.

Pool

Make, say, 8 connections and let requests take turns. Do not close after use. If 8 are not enough, the **pool** makes more. Use a pool in production.



Both give an object with the same **query** function. Put the pool in its own file so any file can import it.

db/pool.js

```
import * as mysql from 'mysql2/promise';
export const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'your-password',
  database: 'sampledb',
});
export default pool;
```

A quick test file, to see what query returns before you wire anything.

tests/test-db.js

```
import pool from '../db/pool.js';
export async function findAll() {
  const subjects = await pool.query('SELECT * FROM subjects');
  return subjects;
}
const [subjects, tableSchema] = await findAll();
await pool.end(); // a test script must close the pool, or it never exits
console.log(subjects);
console.log(tableSchema);
```

SELECT | INSERT | UPDATE | DELETE

```
await pool.query('SELECT * FROM subjects')
```

Index 0

rows

an array of plain objects, one per row

You get

```
[ [ { id: 1, subject_code: 'INT 100',
```

Take index 0. A single match is still an array with one item, and no match is an empty array.

- SELECT gives rows as JavaScript objects already. You do not parse table rows yourself.

- Most of the time take only index 0 and drop the schema: const [rows] = **await** pool.query(...).
- The test script keeps blinking because the pool is still open. Close it with pool.end(). Inside the real app, leave it open for the life of the **server**.

Callback: functions inside functions, so it runs longer.

callback style

```
pool.query(
  'SELECT * FROM students WHERE age > ?', [18],
  (err, results, fields) => {
    if (err) {
      console.error('Query error:', err);
      return;
    }
    console.log('Results:', results);
  }
);
```

Promise + await: shorter, and the one we use. A function that contains await must be async.

async / await

```
async function main() {
  const [rows] = await pool.query('SELECT * FROM students WHERE age > ?', [18]);
  console.log(rows);
}
```

03

Prepared Statements

Your program does not choose the age. The front end sends it. So the value is not known when you write the SQL. A ? marks the empty spot.

- SQL with ? **placeholders** is a **prepared statement**: prepared first, values added later.
- Values go in an array as the second argument. They fill the ? by position: the first value goes in the first ?, and so on. Four ? needs four values.
- You can pass variables, such as the id from the request.

Input `18` `0 OR 1=1`

Glued into the text

```
'SELECT * FROM students WHERE age > '
```

The database receives:

```
SELECT * FROM students WHERE age > 18
```

Works, this time.

Prepared statement

```
pool.query('SELECT * FROM students WHE
```

The command and the value travel separately:

```
? = "18"
```

Same result, and safe.

Why it matters

The slide lists "use prepared statements to prevent **SQL injection**" as a goal. Gluing user input into the SQL text lets that input change the command. A placeholder sends the value separately, so it stays a value. Never build SQL by joining strings with request data.

04

The Repository

Here layers pay off. Only the data access layer changes, from the array to the database. The **service** and the **router** above it stay as they are.

repositories/subject-repository.js

```
import pool from '../db/pool.js';
export async function findAll() {
  const [subjects] = await pool.query('SELECT * FROM subjects');
  return subjects;
}
export async function findById(id) {
  const [subject] = await pool.query('SELECT * FROM subjects WHERE id = ?', [id]);
  if (subject.length == 0) return null;
  return subject;
}
export async function create(subject) {
  const [result] = await pool.query(
    'INSERT INTO subjects (subject_code, subject_title, credit) VALUES (?, ?, ?)',
    [subject.code, subject.title, subject.credit]);
  if (result.affectedRows === 0) return false;
  return findById(result.insertId);
}
export async function update(subject) {
  const [result] = await pool.query(
    'UPDATE subjects SET subject_code = ?, subject_title = ?, credit = ? WHERE id = ?',
    [subject.code, subject.title, subject.credit, subject.id]);
  if (result.affectedRows === 0) return false;
  return findById(subject.id);
}
export async function remove(id) {
  const [result] = await pool.query('DELETE FROM subjects WHERE id = ?', [id]);
  return result.affectedRows > 0;
}
```

- findById still gets an array, even for one row. The code returns that array and no row means null. Take [0] if you want one object.
- create does not send an id. After inserting, it reads the new row back with **insertId**.
- update checks **affectedRows**. 0 means no row had that id.
- The lecturer prefers an update that first looks the row up, returns false if it is missing, and then reads the row again to prove it changed.

> ⚠ **The await ripple**
>
> Once a **repository** function is async, everything that calls it must await it. Forget one and you print a pending **Promise**, or the new row never shows up. Fix it one layer at a time: print in the repository, then the service, then the router, to find where it breaks.

05

Reading the Body and Writing the Response

A POST or PUT carries a JSON **body**. With only the http **module**, you collect the body in **chunks** and parse it yourself. Then you write the JSON answer with one helper.

router.js: getBody and writeResponse

```
function getBody(request) {
  return new Promise((resolve, reject) => {
    let body = '';
    // collect chunks as they arrive, stream by stream
    request.on('data', chunk => { body += chunk.toString(); });
    // the full payload has arrived: parse the text into an object
    request.on('end', () => resolve(body ? JSON.parse(body) : null));
    request.on('error', err => reject(err));
  });
}
async function writeResponse(response, status, message) {
  response.writeHead(status, { 'Content-Type': 'application/json' });
  response.end(JSON.stringify(message));
}
```

router.js: createNewSubject

```
async function createNewSubject(request, response) {
  const subject = await getBody(request, response);
  let message = null;
  let status = 201;
  if (subject) {
    const newSubject = await service.createNewSubject(subject);
    message = newSubject;
    if (!newSubject) {
      status = 409;
      message = { error: `Duplicate subject for id ${subject.id}` };
    }
  } else {
    message = { error: 'Bad request, body is empty' };
    status = 400;
  }
  writeResponse(response, status, message);
}
```

- Empty body: 400 Bad Request. Duplicate: 409 Conflict. Created: 201.
- Not found on update: 404.
- Duplicate subject_code is refused by the database itself with a duplicate entry error. We do not catch that error yet, so the program dies. **Error handling** is a later class.
- Add the Content-Type **header** and print a bit while you **debug**, so an empty reply still tells you something.

06

CRUD over HTTP

Everything lines up: the **endpoint** decides the SQL, the repository runs it, and the **status code** reports the result.

GET /subjects

`SELECT * FROM subjects`

`findAll()` -> 200

GET /subjects/:id

`SELECT ... WHERE id = ?`

`findById(id)` -> 200 or 404

POST /subjects

`INSERT INTO subjects ...`

`create(subject)` -> 201 or 409

PUT /subjects/:id

`UPDATE subjects SET ... WHERE id = ?`

`update(subject)` -> 200 or 404

DELETE /subjects/:id

`DELETE FROM subjects WHERE id = ?`

`remove(id)` -> 204 or 404

- Homework from the lecturer: write the PUT yourself. If create works, update only differs in the SQL and the id.
- Keep the project. The next classes extend the same one.

07

Glossary

affectedRows

How many rows an INSERT, UPDATE or DELETE changed. Use it to see whether the command worked.

async

Start a slow job, keep working, and get called back when it finishes, instead of waiting.

AUTO_INCREMENT

The database counts up the id for you, so you never send one on INSERT and it never duplicates.

await

Pause this async function until a Promise finishes. Only allowed inside an async function.

back end

The part on the server: it handles data, rules and the database, and answers the front end with JSON.

body

The data a request carries after the headers. For an API it is usually JSON.

chunks

Data that arrives in pieces over time instead of all at once.

clients

The side that asks. Usually a browser, but it can be any program or device.

connection

An open line between your program and the database. You need one before you can send any SQL.

CRUD

Create, Read, Update, Delete: the four basic things you do with data.

DBMS

Database management system, such as MySQL. Your back end sends it SQL commands.

debug

Find and fix a bug, usually by stepping through the code and watching the values.

driver

A library that knows how to talk to one kind of database. For MySQL in Node it is mysql2.

endpoint

One method plus path an API answers, such as GET /api/subjects.

Error handling

Catching errors so the program survives and the client gets a useful answer.

event loop

The part of Node that takes requests from the event queue one by one and hands back finished answers.

front end

The part users see and touch. It runs in the browser or on the phone.

header

Extra information about a message, sent as name: value lines, such as Content-Type.

insertId

The id the database gave to the row you just inserted with AUTO_INCREMENT.

JSON

JavaScript Object Notation: a plain text format for data that both humans and programs can read.

localhost

The name for your own machine. localhost:3000 means port 3000 on this computer.

module

A file of code you load into another file with import.

Node

A runtime that runs JavaScript outside the browser, so the same language can write the back end.

placeholders

The ? in a prepared statement. Values fill the ? in order.

pool

A few ready connections kept open and shared. A request borrows one and gives it back, so the database is not flooded.

port

A number that says which program on the machine should get the message. Web servers use 80 by default.

prepared statement

SQL written with ? where the values go, and the values sent separately. The database never mistakes a value for a command.

primary key

The column that identifies each row, usually the id. Update and delete need it.

Promise

An object for a result that is not ready yet. await waits for it.

query

The part of the URL after ?, like page=2&size=10. Used to filter or page a list.

repository
: The layer that reads and writes the database. No business rules, no HTTP.

request
: What the client sends: method, URL, headers and sometimes a body.

router
: Matching an HTTP method and a URL path to the function that handles it.

server
: A program (or the machine running it) that waits for requests and answers them.

service
: The layer that holds the business rules. It calls repositories and knows nothing about HTTP.

SQL injection
: An attack where user input is glued into SQL and changes what the command does. Placeholders prevent it.

status code
: The number that tells the client how it went: 200 OK, 201 Created, 404 Not Found, 500 Server Error.

surrogate key
: A made-up key with no meaning, like a running id. Easier than a real-world code.